Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 8

# Race Conditions and File Locking

Lab 8 guide: watch two buyers oversell a shop, say why, and repair it with a lock.



Heng Rathpisey · Department GIC · Academic year 2025–2026

# Learning objectives

1. Read a short script and find its critical section.
2. Reproduce a race condition, and show which rule it breaks.
3. Protect the whole critical section with a file lock, and test it.

THIS LAB

**Lab 8**

First the concepts, with examples. Then the lab on the server: Core for everyone, Plus and Challenge if you finish early, and a live checkpoint.

DISCUSSION

# Two people buy the last widget at the same moment. Who gets it?

If the shop checks the stock and then sells, what can happen between the check and the sale?

CONCEPTS FIRST

# Critical sections and locks

Three ideas: a check followed by a change, two programs at once, and a lock that makes them take turns.

# A critical section touches shared data

- Shared data is something more than one program can change: here, a stock file.
- A critical section is the part of a program that reads and then changes that shared data.
- While one program is inside, no other program may be inside. This is mutual exclusion.
- The rule must cover the whole sequence: read, decide, write.

FOR THE WHOLE COURSE

## Why learn it here?

Every shared file, database and counter has this problem. The same idea returns in the lectures on semaphores and monitors.

# A race condition

| Step | Buyer A | Buyer B | stock.txt |
|---|---|---|---|
| **1** | reads the stock: 9 | | 9 |
| **2** | | reads the stock: 9 | 9 |
| **3** | decides: 7 is enough | decides: 8 is enough | 9 |
| **4** | writes 9 - 7 = 2 | | 2 |
| **5** | | writes 9 - 8 = 1 | 1 |

Both buyers read 9 before either wrote. 15 widgets are sold, but only 9 existed. The result depends on who is faster: that is a race.

# The rule that must stay true

RULE

## An invariant

A statement that must be true at all times. Here: units sold + units left = the starting stock.

BROKEN

## A race breaks it

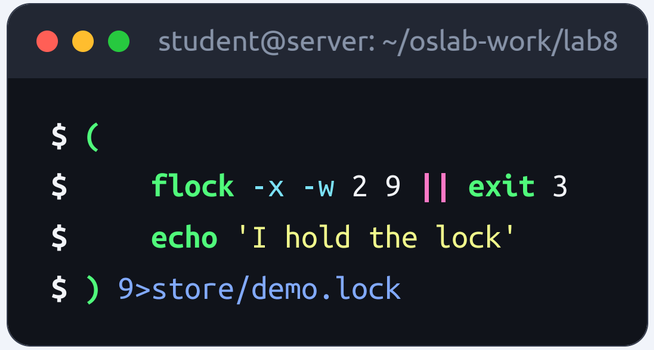
After the race above: 15 sold + 1 left = 16, not 9. The rule is false, so the program is wrong.

IN THE LAB

## Why a delay helps

BUY_DELAY=1 makes the payment take one second. The two buyers then overlap on purpose, so you can see the problem every time.

# A lock with flock




9>store/demo.lock opens the lock file as file descriptor 9 for this group of commands.

flock -x asks for an exclusive lock on it. -w 2 waits at most two seconds. If it cannot get the lock, exit 3 ends the group with status 3.

The lock is released when descriptor 9 is closed, at the closing bracket, or when the process ends for any reason.

# A lock only works if everybody asks for it

## Advisory

flock does not stop a program that never asks. It only makes programs that ask take turns.

## One shared lock file

All buyers must use the same lock file, or they do not meet at the lock.

## Cover the whole sequence

Start the lock before the read, and release it after the last write. A lock around only the write is too late.

LAB 8

# Secure scripts, races and file locking

QuantumTech sells widgets with a small Bash script. It works for one buyer.
Two at once, and it oversells.

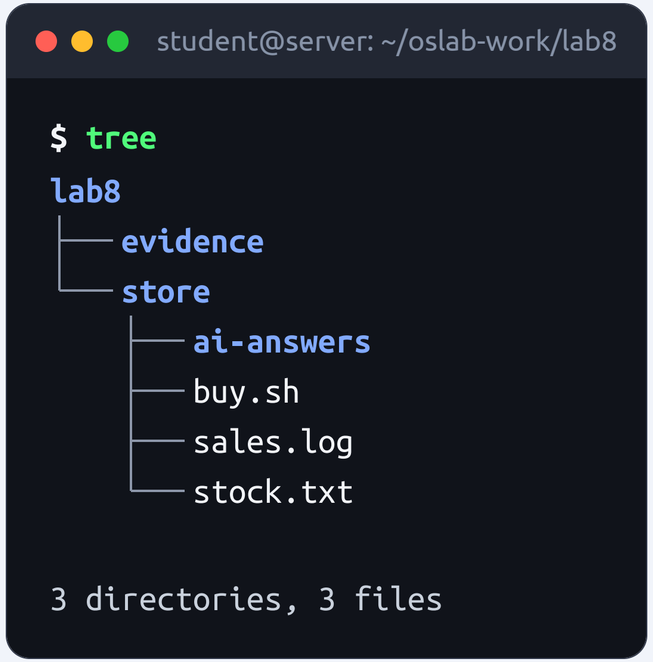
# The scenario

SCENARIO

## QuantumTech's widget shop

The script works for one buyer. On a busy day two buyers arrive together and the shop sells widgets it does not have. You will see the problem, explain it, and repair it.

Your numbers are your own. This guide shows example values.



# Inside the 120 minutes

| Minutes | What happens |
|---|---|
| **0 to 5** | Setup |
| **5 to 20** | Read and trace the script |
| **20 to 30** | Prediction, then class discussion |
| **30 to 50** | Core 1: break it |
| **50 to 75** | Core 2: fix it |

| Minutes | What happens |
|---|---|
| **75 to 90** | Plus and Challenge, or finish Core |
| **90 to 105** | Live checkpoint |
| **105 to 115** | Debrief |
| **115 to 120** | Submit |

# Five helper commands, and why you use them

| Command | What it does | Why |
|---|---|---|
| `oslab values lab8` | Shows your own numbers | Your work is different from your neighbour's |
| `oslab predict lab8` | Saves your prediction once, before the experiment | Guessing first makes you think. The first answer is kept |
| `oslab hint lab8 1` | A hint, levels 1 to 3 | You get unstuck without waiting, at no cost |
| `oslab check lab8` | Tests your script and shows your milestones | You see at once what is still missing |
| `oslab checkpoint lab8` | Opens when the instructor says so, near the end | It shows what you can do alone |

Before the lab, run oslab doctor and oslab prelab lab8. It takes about ten minutes.

# How you work

## Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

## AI tools

Allowed in Core, Plus and Challenge, so test what they tell you. Not allowed in the prediction and the checkpoint.

## Three wrong fixes

In Plus you will see three AI fixes that use flock, look right, and are wrong.

A wrong prediction costs nothing. It is marked for being made and corrected, not for being right.

# Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab8
$ cd "$OSLAB_WORKSPACE/lab8"
$ mkdir -p evidence
$ oslab values lab8
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab8 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab8 prints your own numbers.

# Your values, kept in variables

## 1. The command prints your values

example: your values will be different

```
$ oslab values lab8
Your personal values for lab8 (your-account):
  stock    = 9
  buyer_a  = 7
  buyer_b  = 8
```

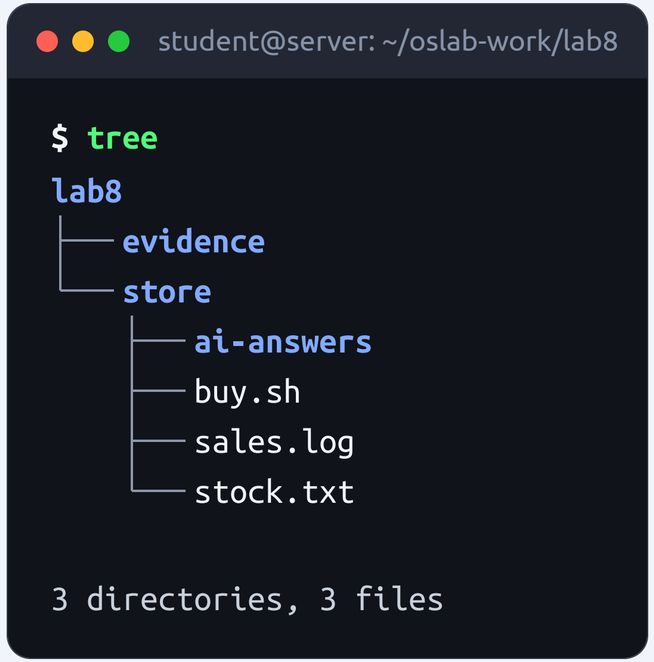
## 2. Type them into shell variables

type your own values

```
$ stock=9
$ buyer_a=7
$ buyer_b=8
```

Write name=value with no spaces around =. In a new terminal, run cd into the lab folder and set them again.

# Your folder




store/buy.sh is the shop script. It works for one buyer.

store/stock.txt holds your starting stock. sales.log is empty.

store/ai-answers/ holds three AI fixes for the Plus task.

READ AND TRACE

# Understand the script first

Fifteen minutes. Do not run two buyers yet.

# Step 1 and 2: one buyer, then put the data back

student@server: ~/oslab-work/lab8

```
$ cat -n store/buy.sh
$ bash store/buy.sh 1
$ cat store/stock.txt store/sales.log

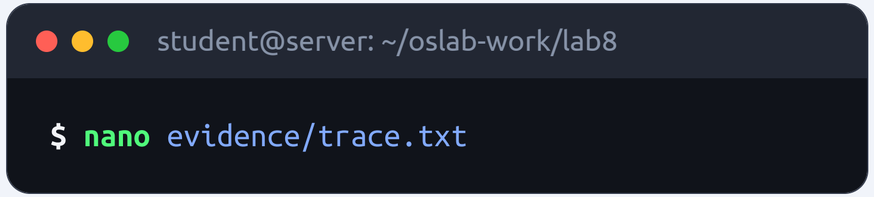
$ printf '%s\n' "$stock" > store/stock.txt
$ : > store/sales.log
```

cat -n shows the script with line numbers. Let one buyer buy one unit and look at the two files.

Then put the test data back. You will use the last two lines many times: one writes your stock, the other empties the log.

# Step 3: answer four questions in evidence/trace.txt

1. Which line reads the stock?
2. Which lines change a file?
3. After which line has the script decided that there is enough stock?
4. Finish the rule: units sold + units left = ...




TRACE

### In your own words

Use line numbers from cat -n. You will use this reading again in Core 2.

PREDICT

# Guess first, then try

Before you run two buyers. Ten minutes.

# The prediction

student@server: ~/oslab-work/lab8

```
$ oslab predict lab8
```

Answer the questions the command asks, about what will happen when two buyers run together.

The class then looks at the spread of answers together, without names.

PREDICTION

### The rules

Alone, and without AI. The first answer is the one that counts. A wrong guess costs nothing.

# Break it

Twenty minutes, from minute 30 to 50. Make two buyers run together.

# Step 1: two buyers at the same time

student@server: ~/oslab-work/lab8

```
$ BUY_DELAY=1 bash store/buy.sh "$buyer_a" > evidence/a.txt 2>&1 &
$ pid_a=$!
$ BUY_DELAY=1 bash store/buy.sh "$buyer_b" > evidence/b.txt 2>&1 &
$ pid_b=$!
$ wait "$pid_a"; rc_a=$?
$ wait "$pid_b"; rc_b=$?
```

A trailing & starts a command in the background, so the next line runs at once. $! is the PID of the last background command.

wait PID waits for that one command and $? then holds its exit status. BUY_DELAY=1 makes the payment take a second, so the two buyers overlap.

# Step 2: save what happened

student@server: ~/oslab-work/lab8

```
$ {
$   echo "start=$stock A wants $buyer_a (exit $rc_a) B wants $buyer_b (exit $rc_b)"
$   cat evidence/a.txt evidence/b.txt
$   echo "stock.txt now: $(cat store/stock.txt)"
$   cat store/sales.log
$ } | tee evidence/race.txt
```

The braces group several commands, so tee saves the output of all of them in evidence/race.txt.

$(...) puts the output of a command into the text.

# Step 3 to 5: check the rule, then choose your own quantities

## Check the rule

- How many units were sold? How many are left?
- Add one line to evidence/race.txt that says which number is wrong.

## Your decision

- Put the test data back. Choose two small quantities whose sum is less than your stock.
- Run the two buyers again. Is stock.txt correct now? Add your quantities, the result and one sentence.
- Run oslab check lab8: two milestones pass and one does not. That is expected.

THINK

### Why this matters

With small quantities the race can hide. A passing run does not show that the script is safe.

CORE 2

# Fix it

Twenty-five minutes, from minute 50 to 75. You decide where the lock goes.

# Step 1 to 3: keep a copy, try flock, plan the lock

student@server: ~/oslab-work/lab8

```
$ cp -- store/buy.sh store/buy_unsafe.sh
$ (
$     flock -x -w 2 9 || exit 3
$     echo 'I hold the lock'
$ ) 9>store/demo.lock
```

Keep the unsafe script. You will compare against it later.

Try flock alone first, as on the concepts slide.

PLAN

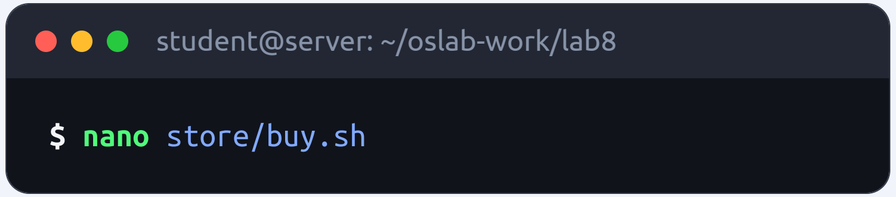
### Before you edit

In evidence/trace.txt write the first line and the last line of buy.sh that must be inside the lock, and why.

# Step 4: edit buy.sh and add the lock

### You decide

- Where the lock starts.
- What the script does when it cannot get the lock: print a message and exit with status 3.
- Which lock file all buyers share.
- Keep the BUY_DELAY line.




YOUR TURN

### You write it

Nobody gives you the lines. Use your plan from the trace.

# Step 5 and 6: test it again

1. Put the test data back.
2. Run the two buyers from Core 1 again.
3. Save the result in evidence/locked.txt, with the same block as before.
4. Run oslab check lab8 until all three milestones pass.

DONE

### Core is complete when

oslab check lab8 shows milestones: 3/3, and your trace names the lines inside the lock.

HINTS

### Stuck for five minutes?

oslab hint lab8 1, then 2, then 3. Hints cost no points.

# Plus: review three AI answers

student@server: ~/oslab-work/lab8

```
$ diff store/buy_unsafe.sh store/ai-answers/a.sh

$ mkdir -p try
$ cp -- store/ai-answers/a.sh try/buy.sh
$ printf '%s\n' "$stock" > try/stock.txt
$ : > try/sales.log
$ BUY_DELAY=1 bash try/buy.sh "$buyer_a" & BUY_DELAY=1 bash try/buy.sh "$buyer_b" & wait
$ cat try/stock.txt try/sales.log
```

Each of the three answers uses flock, and each is wrong. Test them in a separate folder, so your own script stays safe.

PLUS

**For each answer, write in evidence/ai-review.txt**

The line that is wrong, and one sentence on why two buyers can still be inside the critical section together.

# Challenge: choose one

CHALLENGE 1

## Killed buyer

Start one buyer of your fixed script with BUY_DELAY=20 in the background, save its PID, and kill that PID while it holds the lock. Does the next buyer wait for ever? Explain what the kernel does with the lock.

CHALLENGE 2

## Smallest repair

Make ai-answers/a.sh correct by moving lines only. Do not add or delete a line.

Save your commands and your explanation in evidence/challenge.txt.

LIVE CHECKPOINT

# Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

# Why is there a checkpoint?

### It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

### Your numbers are new

The questions use new numbers, made for you at that moment. An answer copied from a neighbour does not fit.

### The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

### It is fair

Everyone gets the same kind of task. You can answer the questions even if Core is not finished.

# What happens, step by step

**1. The instructor opens it**
Close your AI tools

→

**2. oslab checkpoint lab8**
Answer the questions

→

**3. Change buy.sh**
The small change the command prints

→

**4. oslab check lab8**
A fourth milestone must pass

## Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

## While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

## When you finish

oslab check lab8 shows whether the checkpoint milestone passes. You can fix your script and check again.

# What the checkpoint asks

| Part | What you do |
| --- | --- |
| **Three numbers, with new values** | Three buyers run your repaired buy.sh together. How many purchases are accepted, what does stock.txt hold, how many units does sales.log show? |
| **One sentence** | Which line of your script makes a later buyer see the earlier buyer's update? |
| **One change to buy.sh** | One purchase may be at most a limit. A larger quantity prints a message, exits with status 4 and changes nothing. Then run oslab check lab8. |

## Can I answer again?

No. Your answers are saved once. You can still fix your script and run oslab check lab8 again.

## Core not finished?

Answer the questions first. They do not depend on a finished script.

# Look back, then hand in

Ten minutes of debrief, five to submit.

# Debrief: look at your prediction again

student@server: ~/oslab-work/lab8

```
$ cat "$OSLAB_WORKSPACE/.records/lab8-predict.json"
```

For each answer write confirmed or corrected.

ANSWER

### Explain in three to five sentences

Why can the stock stay above zero while the shop sells too much? Which lines must share one lock?

# Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab8/evidence"
$ cd "$OSLAB_WORKSPACE/lab8"
$ cp -- store/buy.sh "$SUBMISSION_REPO/lab8/"
$ cp -- evidence/trace.txt evidence/race.txt evidence/locked.txt "$SUBMISSION_REPO/lab8/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Add ai-review.txt or challenge.txt if you did them. Do not delete store/stock.lock while a buyer is running.

# Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab8
├── buy.sh
├── evidence
│   ├── locked.txt
│   ├── race.txt
│   └── trace.txt
└── README.md

1 directory, 5 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

# How it is marked: 10 points

| Evidence | Points |
|---|---|
| **A working script: oslab check lab8 passes the three Core milestones** | 2 |
| **Tests: the unsafe run, your own quantities, and the run after the repair** | 2 |
| **The prediction saved in time, and an honest confirmed or corrected explanation** | 2 |
| **The live checkpoint: answers, the change to your script, and your sentence** | 3 |
| **Clear, complete evidence files and report** | 1 |

Plus and Challenge are not needed for full marks. They are noted in your feedback. Your prediction is marked for being made and corrected, not for being right.

# Common mistakes

| Mistake | What you see | Fix |
|---|---|---|
| **Lock around only the write** | Both buyers still read the same stock | Start the lock before the read |
| **Each buyer uses its own lock file** | No waiting, and the race stays | All buyers share one lock file |
| **Forgetting the exit code on a failed lock** | The script goes on without the lock | flock ... \|\| exit 3 |
| **Testing with tiny quantities** | A race that does not show | Use quantities that make the sum exceed the stock |

BEYOND THE CORE

# Conservation and the drop box

Optional extensions. At most two buyers per run, and no stress loops on the shared server.

Lab 8

# Check the invariant with numbers

student@server: ~/oslab-work/lab8

```
$ awk '$1 == "sold" {sum += $2} END {print "sold units:", sum+0}' store/sales.log
sold units: 7
$ cat store/stock.txt
2
```

Add the units sold. Do not only count the lines of the log.

Check: starting stock = final stock + accepted sold units.

Try quantities whose sum exceeds the stock, and an invalid stock such as abc in an owned copy. No sale may be logged.

Advisory locks only work when every writer takes part, and a crash between the two writes can still lose data. This is a teaching script, not a database.

# A prepared drop box, and the log

DROP BOX

## A shared drop box

Only if the instructor prepared a narrow folder. Check its parent folders, its group or ACL and its naming rule before you send one test request.

LOG

## Log housekeeping

Copy sales.log to a uniquely named evidence file before you start a new case. Never rotate a log while a buyer is running.

CAREFUL

## Clean only what is yours

Remove only known demonstration files you own, never files that match a wildcard in a classmate's folder.

# Key takeaways

01

### Critical section

The read, the decision and the write belong together. Protect all of them.

02

### A race is about timing

It may pass many times and then fail. A delay makes it visible, it does not cause it.

03

### flock

An advisory lock on a shared file. Every program must ask, and all must use the same file.

04

### Test the fix

Use the same run as before, and check the rule: sold plus left equals the start.

# Next session

COMING UP

## Lab 9: a vault deadlock

Two workers each hold one lock and wait for the other. Find the order that avoids it.

SOURCE

## Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 6, Synchronization Tools.